

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

Лабораторная работа № 2
по дисциплине «Анализ алгоритмов»

Тема Сравнительный анализ алгоритмов умножения матриц

Студент Крылов В.С.

Группа ИУ7-52Б

Преподаватели Волкова Л.Л.

Преподаватели Строганов Д.В.

Москва, 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **4**

1 Аналитическая часть **5**
1.1 Постановка задачи 5
1.2 Вывод 5

2 Конструкторская часть **6**
2.1 Требования к программному обеспечению 6
2.2 Разработка алгоритмов 6
2.2.1 Стандартный алгоритм умножения матриц 7
2.2.2 Алгоритм Винограда 9
2.2.3 Оптимизированный алгоритм Винограда 14
2.3 Вывод 19

3 Технологическая часть **20**
3.1 Средства разработки 20
3.2 Реализация алгоритмов 20
3.3 Тестирование реализаций 23
3.4 Вывод 23

4 Исследовательская часть **24**
4.1 Модель вычислений 24
4.2 Оценка трудоёмкости алгоритмов 25
4.2.1 Стандартный алгоритм умножения матриц 25
4.2.2 Базовый алгоритм Винограда 25
4.2.3 Оптимизированный алгоритм Винограда 27
4.2.4 Сводная аналитическая оценка 29
4.3 Характеристики устройства 29
4.4 Инструменты замера процессорного времени 30
4.5 Результаты массированных замеров 30
4.6 Сравнительный анализ 32
4.7 Вывод 33

ЗАКЛЮЧЕНИЕ **34**

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ **35**

ВВЕДЕНИЕ

Целью лабораторной работы является выполнение сравнительного анализа алгоритмов умножения матриц по трудоёмкости.

Задачи работы:

— описать решаемую задачу согласно варианту;

— разработать алгоритмы решения задачи согласно варианту: стандартный алгоритм умножения матриц, алгоритм Винограда и оптимизированный по варианту алгоритм Винограда;

— описать разработанные алгоритмы схемами алгоритмов;

— описать требования к программному обеспечению;

— описать средства реализации алгоритмов;

— реализовать разработанные алгоритмы;

— выполнить тестирование реализации алгоритмов;

— описать модель вычислений для оценки трудоёмкости алгоритмов и выполнить теоретический расчёт трудоёмкости;

— описать инструменты замера процессорного времени выполнения реализаций алгоритмов;

— выполнить массированные замеры процессорного времени выполнения реализаций алгоритмов умножения матриц в зависимости от линейного размера квадратных матриц, отдельно для чётных и нечётных размеров;

— выполнить сравнительный анализ алгоритмов по трудоёмкости и по данным выполненных замеров.

1 Аналитическая часть

1.1 Постановка задачи

Даны матрицы A размера $M \times N$ и B размера $N \times Q$, содержащие целочисленные элементы. Требуется вычислить результирующую матрицу $C = A \times B$ размера $M \times Q$, элементы которой определяются соотношением:

$$C_{i,j} = \sum_{k=0}^{N-1} A_{i,k} \cdot B_{k,j}, \quad i = 0, \ldots, M-1, \quad j = 0, \ldots, Q-1. \tag{1.1}$$

Операция умножения матриц определена тогда и только тогда, когда количество столбцов матрицы A совпадает с количеством строк матрицы B.

Для решения поставленной задачи требуется разработать и программно реализовать следующие алгоритмы:

— стандартный алгоритм умножения матриц;

— базовый алгоритм Винограда;

— оптимизированный по варианту алгоритм Винограда.

Требуется сравнить разработанные реализации между собой по теоретической трудоёмкости и по данным замеров процессорного времени их выполнения на квадратных матрицах ($M = N = Q$), с раздельным рассмотрением для чётных и нечётных линейных размеров матриц.

Входные данные: линейные размеры матриц $M, N, Q \in \mathbb{N}$ (для квадратных матриц — линейный размер N) и целочисленные элементы матриц $A_{i,k}, B_{k,j} \in \mathbb{Z}$. Выходные данные: матрица C размера $M \times Q$ ($N \times N$ для квадратных матриц), элементы которой $C_{i,j} \in \mathbb{Z}$.

1.2 Вывод

В результате аналитической части была формализована задача умножения матриц, определены форматы входных и выходных данных, а также сформирован перечень алгоритмов, подлежащих разработке, реализации и сравнительному анализу.

2 Конструкторская часть

2.1 Требования к программному обеспечению

К разрабатываемому программному обеспечению предъявляются следующие требования:

1. ввод линейных размеров матриц M, N, Q (натуральные числа);
2. ввод целочисленных элементов матриц A и B вручную с клавиатуры либо автоматическая случайная генерация элементов в заданном числовом диапазоне;
3. вывод текущих матриц A и B в консоль в форматированном виде;
4. циклический вывод интерактивного меню с возможностью выбора и выполнения следующих операций:
   - 4.1. вычисление произведения матриц с использованием стандартного алгоритма;
   - 4.2. вычисление произведения матриц с использованием базового алгоритма Винограда;
   - 4.3. вычисление произведения матриц с использованием оптимизированного по варианту алгоритма Винограда;
   - 4.4. перегенерация исходных матриц случайными значениями;
   - 4.5. повторный ввод размерностей и элементов матриц пользователем;
   - 4.6. вывод текущих матриц на экран;
   - 4.7. запуск серии массированных замеров процессорного времени выполнения всех трёх алгоритмов для квадратных матриц чётных и нечётных размеров с выгрузкой результатов в CSV-файлы;
   - 4.8. завершение работы программы.

2.2 Разработка алгоритмов

Для каждого из разрабатываемых алгоритмов умножения определены:

— A — входная матрица размера $M \times N$ с целочисленными элементами;

— B — входная матрица размера $N \times Q$ с целочисленными элементами;

— C — результирующая матрица размера $M \times Q$, элементы которой перед началом вычислений инициализируются нулями;

— M, N, Q — линейные размеры перемножаемых матриц.

Для базового и оптимизированного алгоритмов Винограда дополнительно определены:

— row — вспомогательный вектор длины M для промежуточных сумм парных произведений строк матрицы A;

— col — вспомогательный вектор длины Q для промежуточных сумм парных произведений столбцов матрицы B;

Для оптимизированного алгоритма Винограда дополнительно определены:

— $half_n$ — кэшированная граница диапазонов циклов $\lfloor N/2 \rfloor$;

— two_k — локальная переменная, хранящая вычисленный сдвигом индекс $k \ll 1$ ($2k$).

2.2.1 Стандартный алгоритм умножения матриц

В стандартном алгоритме производится вычисление каждого элемента $C_{i,j}$ результирующей матрицы по математическому определению произведения матриц [1]:

$$C_{i,j} = \sum_{k=0}^{N-1} A_{i,k} \cdot B_{k,j}, \quad i = 0, \ldots, M-1, \quad j = 0, \ldots, Q-1. \tag{2.1}$$

Алгоритм состоит из трёх вложенных циклов: внешний цикл организует проход по строкам $i \in [0, M-1]$, средний — по столбцам $j \in [0, Q-1]$, а внутренний выполняет накопление суммы произведений по индексу $k \in [0, N-1]$. Схема стандартного алгоритма умножения матриц приведена на рисунке 2.1.

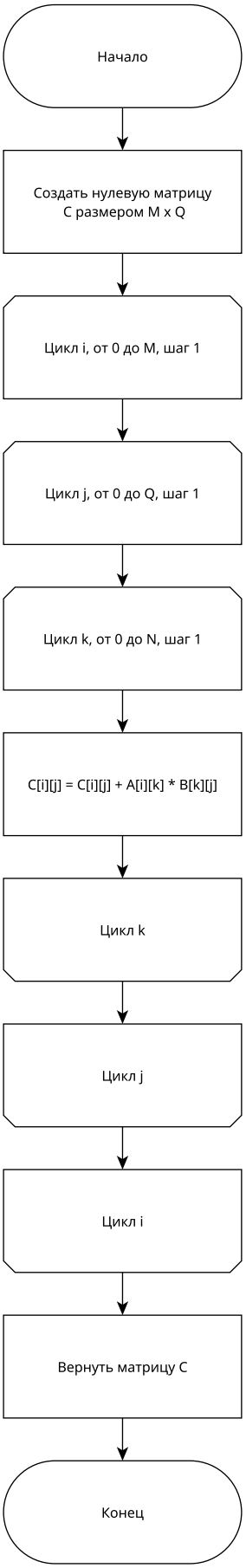


Рисунок 2.1 — Схема стандартного алгоритма умножения матриц

2.2.2 Алгоритм Винограда

В алгоритме Винограда уменьшается общее число умножений элементов матриц ценой увеличения количества операций сложения [2]. Вычисление произведения основано на предварительном расчёте двух одномерных массивов, зависящих только от строк матрицы A и столбцов матрицы B соответственно:

$$row_i = \sum_{k=0}^{\lfloor N/2 \rfloor - 1} A_{i,2k} \cdot A_{i,2k+1}, \quad i = 0, \ldots, M-1, \tag{2.2}$$

$$col_j = \sum_{k=0}^{\lfloor N/2 \rfloor - 1} B_{2k,j} \cdot B_{2k+1,j}, \quad j = 0, \ldots, Q-1. \tag{2.3}$$

После вычисления массивов row и col каждый элемент матрицы результата инициализируется значением $-row_i - col_j$, после чего в основном цикле суммируются произведения парных сумм элементов:

$$C_{i,j} = -row_i - col_j + \sum_{k=0}^{\lfloor N/2 \rfloor - 1} (A_{i,2k} + B_{2k+1,j}) \cdot (A_{i,2k+1} + B_{2k,j}). \tag{2.4}$$

Если общий размер N является нечётным числом, последний непарный элемент строки и столбца учитывается отдельной коррекцией:

$$C_{i,j} = C_{i,j} + A_{i,N-1} \cdot B_{N-1,j}. \tag{2.5}$$

Схема базового алгоритма Винограда приведена на рисунках 2.2 - 2.5.

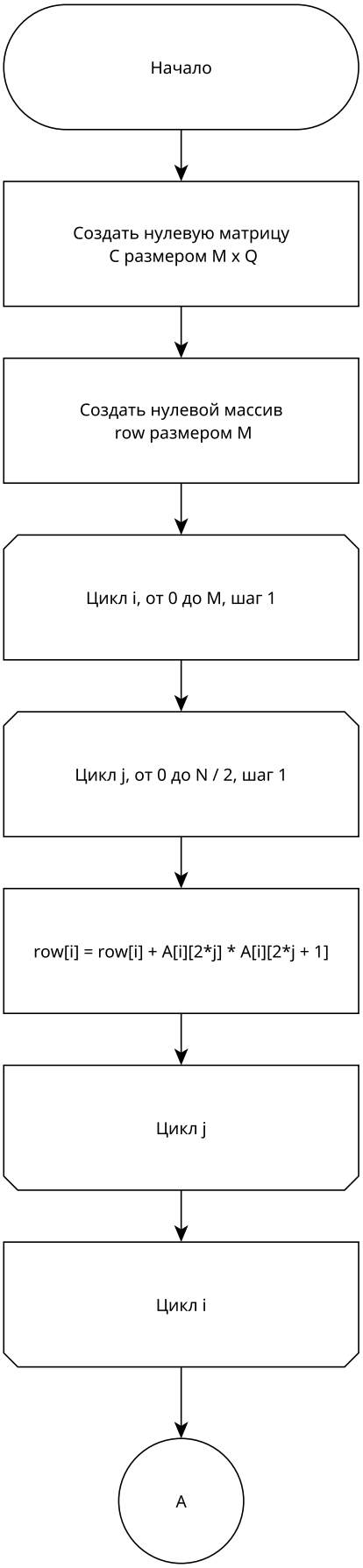


Рисунок 2.2 — Схема базового алгоритма Винограда

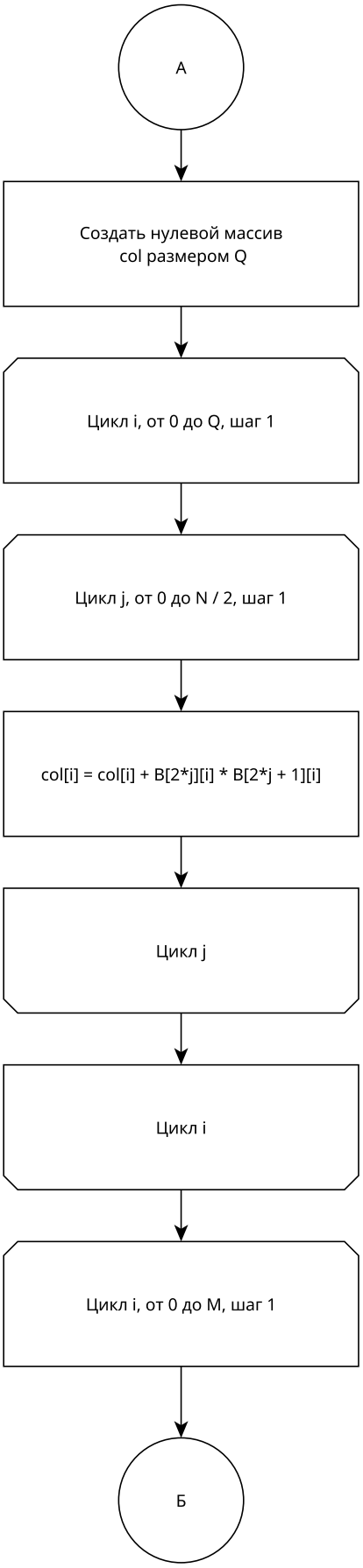


Рисунок 2.3 — Схема продолжения базового алгоритма Винограда

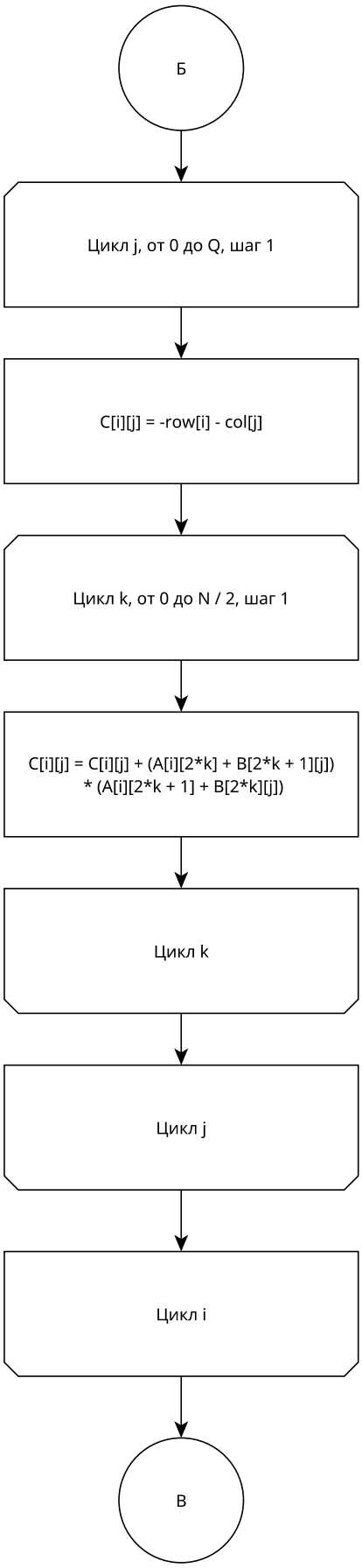


Рисунок 2.4 — Схема продолжения базового алгоритма Винограда

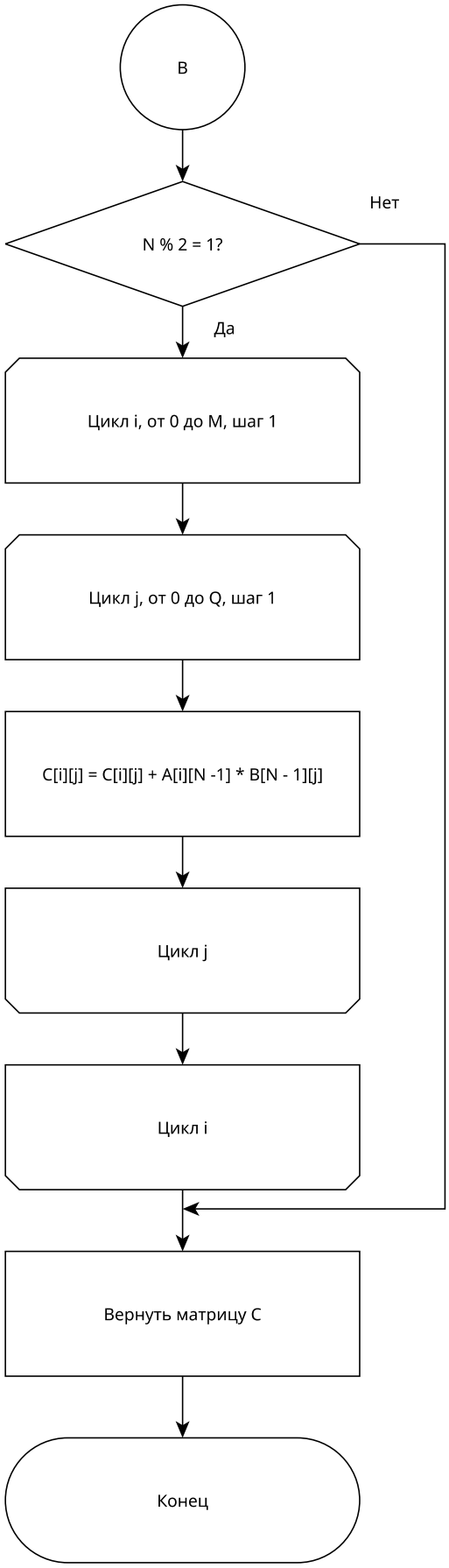


Рисунок 2.5 — Схема продолжения базового алгоритма Винограда

2.2.3 Оптимизированный алгоритм Винограда

В соответствии с индивидуальным вариантом к алгоритму Винограда применены следующие две оптимизации:

1. замена умножения $2 \cdot k$ при индексации побитовым сдвигом влево на один разряд: $k \ll 1$. Значение вычисляется один раз в начале каждой итерации цикла по k, сохраняется в локальную переменную two_k и затем повторно используется во всех обращениях к элементам текущей итерации;
2. предварительное вычисление (кэширование) значения верхней границы циклов $half_n = \lfloor N/2 \rfloor$ в отдельную константу до входа в циклы вместо многократного повторения целочисленного деления $N/2$ при каждой проверке условия продолжения цикла.

Применение данных оптимизаций не изменяет результирующие значения матрицы C и общее количество операций над её элементами, однако сокращает накладные расходы на целочисленную арифметику внутри циклов. Схемы оптимизированного алгоритма Винограда приведена на рисунках 2.6 - 2.10.

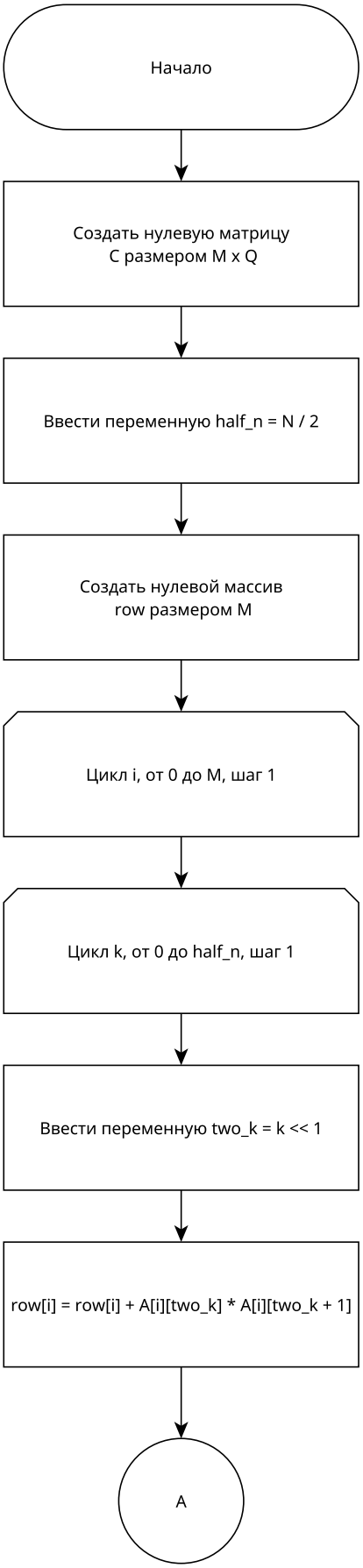


Рисунок 2.6 — Схема оптимизированного алгоритма Винограда

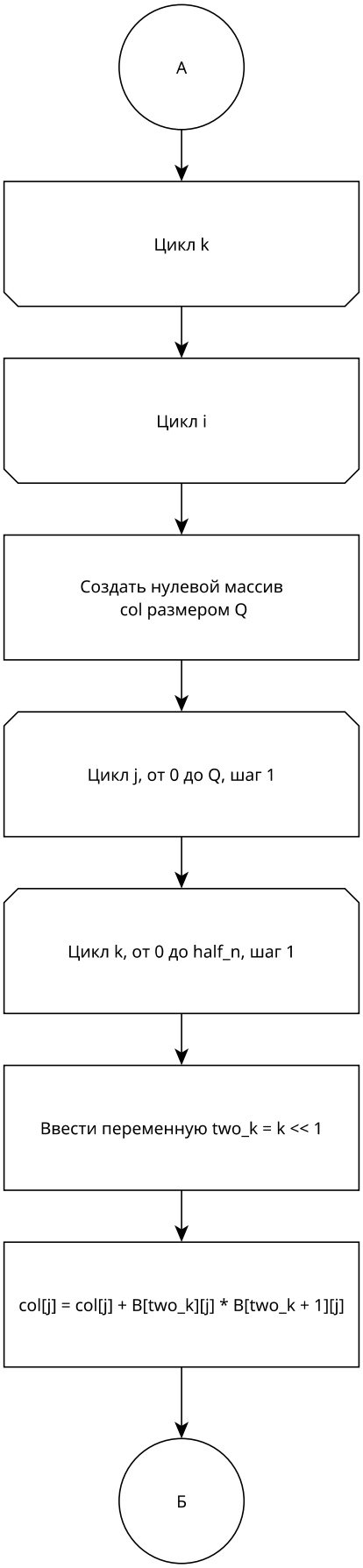


Рисунок 2.7 — Схема продолжения оптимизированного алгоритма Винограда

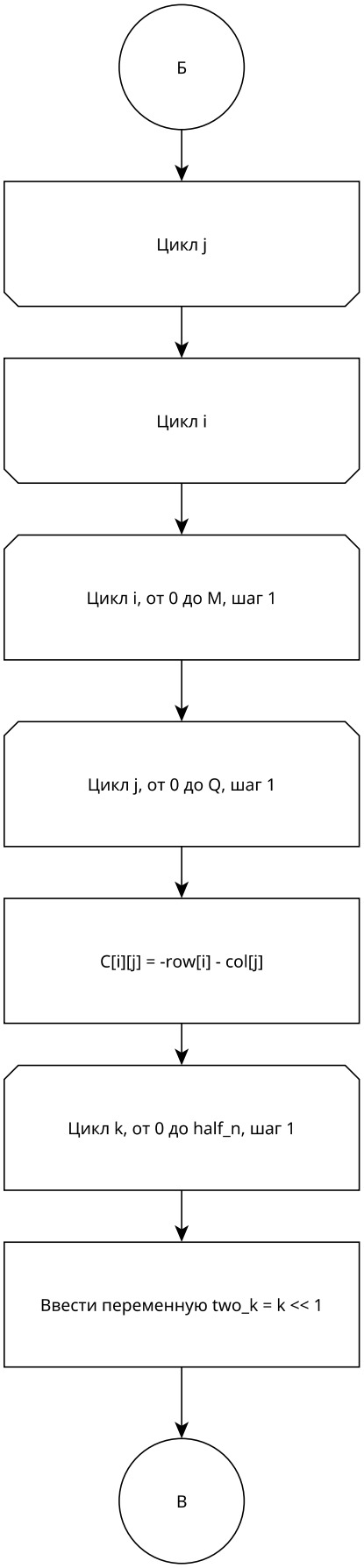


Рисунок 2.8 — Схема продолжения оптимизированного алгоритма Винограда

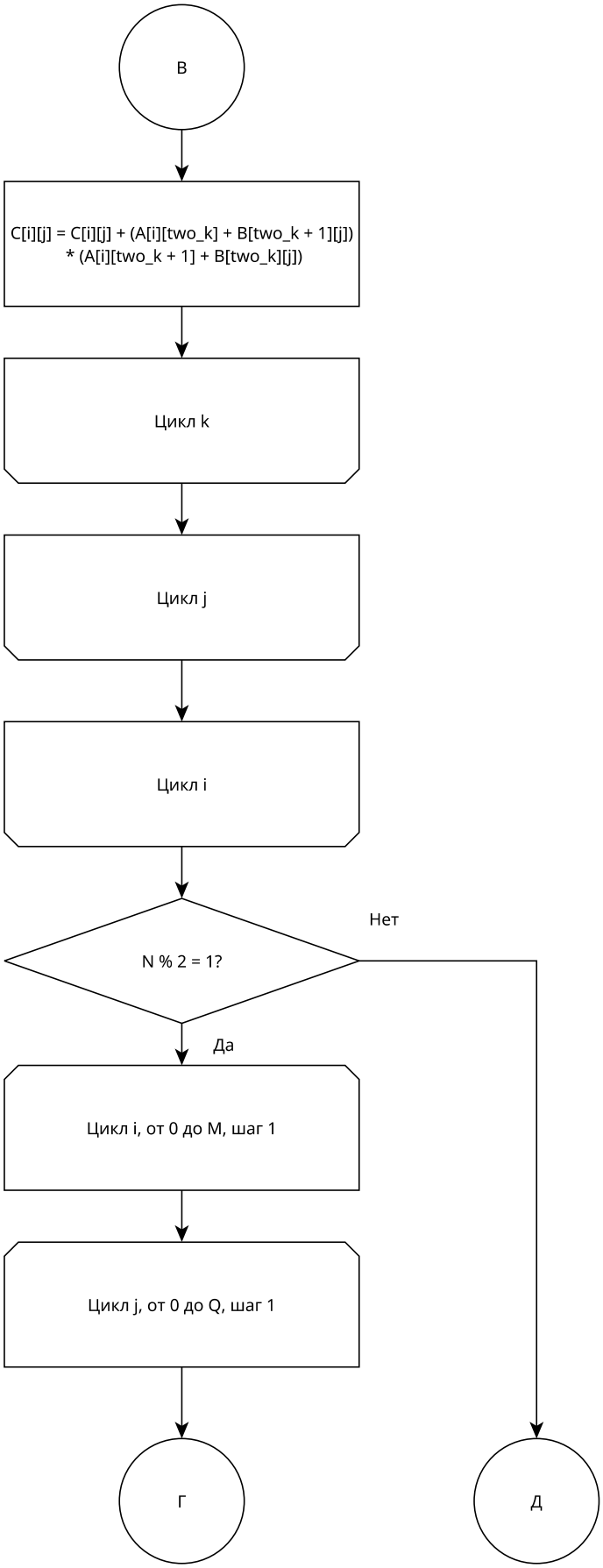


Рисунок 2.9 — Схема продолжения оптимизированного алгоритма Винограда

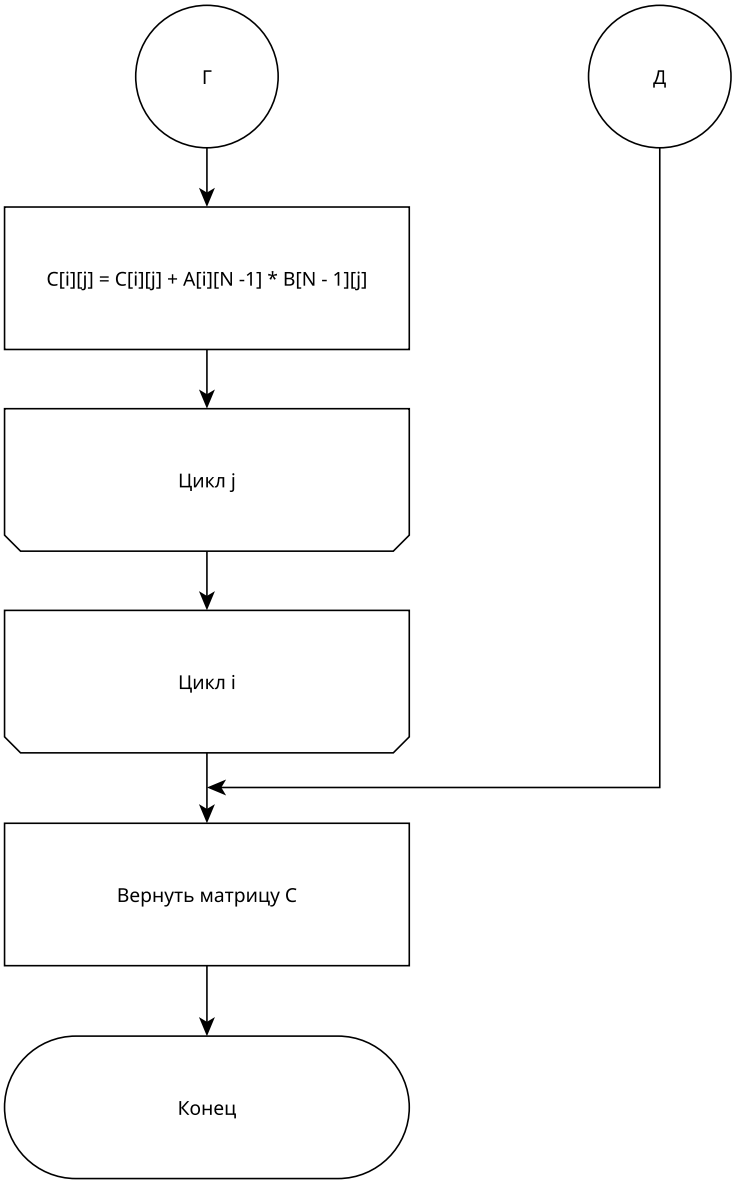


Рисунок 2.10 — Схема продолжения оптимизированного алгоритма Винограда

2.3 Вывод

В результате выполнения конструкторской части были формализованы требования к разрабатываемому программному обеспечению, описаны математические основы и спецификации входных и выходных данных для стандартного алгоритма умножения матриц, базового алгоритма Винограда и его оптимизированной модификации, а также представлены схемы данных алгоритмов.

3 Технологическая часть

3.1 Средства разработки

В качестве языка программирования со статической типизацией был выбран язык C++ (стандарт C++20) [3]. В процессе разработки использовались следующие средства стандартной библиотеки языка:

— контейнер `std::vector<std::vector<int>>` — для динамического представления целочисленных матриц;

— контейнер `std::vector<int>` — для хранения вспомогательных массивов row и col;

— функция `clock()` из заголовочного файла `<ctime>` — для замера процессорного времени работы исследуемых алгоритмов [4];

Сборка программы осуществлялась компилятором GCC с флагами компиляции:

```
-std=c++20 -Wall -Wextra -Wpedantic -O0
```

Флаг оптимизации `-O0` задан принудительно, чтобы компилятор не выполнял автоматическую оптимизацию.

3.2 Реализация алгоритмов

На рисунке 3.1 представлена программная реализация стандартного алгоритма умножения матриц.

```
Matrix multiply_standard(const Matrix &A, const Matrix &B, size_t M, size_t N,
    size_t Q)
{
    Matrix C = create_matrix(M, Q, 0);

    for (size_t i = 0; i < M; ++i)
        for (size_t j = 0; j < Q; ++j)
            for (size_t k = 0; k < N; ++k)
                C[i][j] += A[i][k] * B[k][j];

    return C;
}
```

Рисунок 3.1 — Стандартный алгоритм умножения матриц

На рисунке 3.2 представлена реализация базового алгоритма Винограда.

```
Matrix multiply_winograd(const Matrix &A, const Matrix &B, size_t M, size_t N,
    size_t Q)
{
    Matrix C = create_matrix(M, Q, 0);

    vector<int> row(M, 0);
    for (size_t i = 0; i < M; ++i)
        for (size_t j = 0; j < N / 2; ++j)
            row[i] += A[i][2 * j] * A[i][2 * j + 1];

    vector<int> col(Q, 0);
    for (size_t j = 0; j < Q; ++j)
        for (size_t i = 0; i < N / 2; ++i)
            col[j] += B[2 * i][j] * B[2 * i + 1][j];

    for (size_t i = 0; i < M; ++i)
    {
        for (size_t j = 0; j < Q; ++j)
        {
            C[i][j] = -row[i] - col[j];
            for (size_t k = 0; k < N / 2; ++k)
                C[i][j] += (A[i][2 * k] + B[2 * k + 1][j]) * (A[i][2 * k + 1] +
                    B[2 * k][j]);
        }
    }

    if (N % 2 == 1)
        for (size_t i = 0; i < M; ++i)
            for (size_t j = 0; j < Q; ++j)
                C[i][j] += A[i][N - 1] * B[N - 1][j];

    return C;
}
```

Рисунок 3.2 — Базовый алгоритм Винограда

На рисунке 3.3 представлена реализация оптимизированного алгоритма Винограда.

```
Matrix multiply_winograd_opt(const Matrix &A, const Matrix &B, size_t M, size_t
    N, size_t Q)
{
    Matrix C = create_matrix(M, Q, 0);
    const size_t half_n = N / 2;
    size_t two_k;

    vector<int> row(M, 0);
    for (size_t i = 0; i < M; ++i)
    {
        for (size_t k = 0; k < half_n; ++k)
        {
            two_k = k << 1;
            row[i] += A[i][two_k] * A[i][two_k + 1];
        }
    }

    vector<int> col(Q, 0);
    for (size_t j = 0; j < Q; ++j)
    {
        for (size_t k = 0; k < half_n; ++k)
        {
            two_k = k << 1;
            col[j] += B[two_k][j] * B[two_k + 1][j];
        }
    }

    for (size_t i = 0; i < M; ++i)
    {
        for (size_t j = 0; j < Q; ++j)
        {
            C[i][j] = -row[i] - col[j];
            for (size_t k = 0; k < half_n; ++k)
            {
                two_k = k << 1;
                C[i][j] += (A[i][two_k] + B[two_k + 1][j]) * (A[i][two_k + 1] +
                    B[two_k][j]);
            }
        }
    }

    if (N % 2 == 1)
        for (size_t i = 0; i < M; ++i)
            for (size_t j = 0; j < Q; ++j)
                C[i][j] += A[i][N - 1] * B[N - 1][j];

    return C;
}
```

Рисунок 3.3 — Оптимизированный алгоритм Винограда

3.3 Тестирование реализаций

В таблице 3.1 представлены результаты функционального тестирования разработанных реализаций алгоритмов умножения матриц.

Таблица 3.1 — Функциональные тесты

№	Входные данные		Выходные данные		Комментарий
	Матрица A	**Матрица** B	**Ожид.**	**Получ.**	
1	$\begin{pmatrix}3\end{pmatrix}$	$\begin{pmatrix}-4\end{pmatrix}$	$\begin{pmatrix}-12\end{pmatrix}$	$\begin{pmatrix}-12\end{pmatrix}$	Матрицы размера 1×1
2	$\begin{pmatrix}1&2&3\\4&5&6\\7&8&9\end{pmatrix}$	$\begin{pmatrix}1&0&0\\0&1&0\\0&0&1\end{pmatrix}$	$\begin{pmatrix}1&2&3\\4&5&6\\7&8&9\end{pmatrix}$	$\begin{pmatrix}1&2&3\\4&5&6\\7&8&9\end{pmatrix}$	Умножение на единичную матрицу $E_{3\times3}$
3	$\begin{pmatrix}1&2\\3&4\end{pmatrix}$	$\begin{pmatrix}0&0\\0&0\end{pmatrix}$	$\begin{pmatrix}0&0\\0&0\end{pmatrix}$	$\begin{pmatrix}0&0\\0&0\end{pmatrix}$	Умножение на нулевую матрицу
4	$\begin{pmatrix}1&2\\3&4\end{pmatrix}$	$\begin{pmatrix}5&6\\7&8\end{pmatrix}$	$\begin{pmatrix}19&22\\43&50\end{pmatrix}$	$\begin{pmatrix}19&22\\43&50\end{pmatrix}$	Квадратные матрицы 2×2, чётный размер
5	$\begin{pmatrix}1&2&3\\4&5&6\\7&8&9\end{pmatrix}$	$\begin{pmatrix}9&8&7\\6&5&4\\3&2&1\end{pmatrix}$	$\begin{pmatrix}30&24&18\\84&69&54\\138&114&90\end{pmatrix}$	$\begin{pmatrix}30&24&18\\84&69&54\\138&114&90\end{pmatrix}$	Квадратные матрицы 3×3, нечётный размер
6	$\begin{pmatrix}1&2&3\\4&5&6\end{pmatrix}$	$\begin{pmatrix}7&8\\9&10\\11&12\end{pmatrix}$	$\begin{pmatrix}58&64\\139&154\end{pmatrix}$	$\begin{pmatrix}58&64\\139&154\end{pmatrix}$	Прямоугольные матрицы 2×3 и 3×2
7	$\begin{pmatrix}1&2\\3&4\\5&6\end{pmatrix}$	$\begin{pmatrix}1&2&3&4\\5&6&7&8\end{pmatrix}$	$\begin{pmatrix}11&14&17&20\\23&30&37&44\\35&46&57&68\end{pmatrix}$	$\begin{pmatrix}11&14&17&20\\23&30&37&44\\35&46&57&68\end{pmatrix}$	Прямоугольные матрицы 3×2 и 2×4
8	$\begin{pmatrix}1&2&3&4\end{pmatrix}$	$\begin{pmatrix}1\\2\\3\\4\end{pmatrix}$	$\begin{pmatrix}30\end{pmatrix}$	$\begin{pmatrix}30\end{pmatrix}$	Вектор-строка на вектор-столбец
9	$\begin{pmatrix}1\\2\\3\end{pmatrix}$	$\begin{pmatrix}4&5\end{pmatrix}$	$\begin{pmatrix}4&5\\8&10\\12&15\end{pmatrix}$	$\begin{pmatrix}4&5\\8&10\\12&15\end{pmatrix}$	Вектор-столбец на вектор-строку

Все тесты были пройдены успешно без расхождений в результатах.

3.4 Вывод

В результате выполнения технологической части были обоснованы средства реализации, приведены листинги исходного кода стандартного, базового и оптимизированного алгоритмов Винограда на языке C++, а также разработана система тестов, подтвердившая корректность всех разработанных реализаций.

4 Исследовательская часть

4.1 Модель вычислений

Для оценки трудоёмкости алгоритмов умножения матриц используется модель вычислений, основанная на подсчёте элементарных операций:

1. операции присваивания, сложения, вычитания (включая унарные знаки), побитового сдвига, взятия индекса, инкремента и сравнения ($=$, $+$, $-$, $+=$, $-=$, $\ll$, $\gg$, $[]$, $++$, $--$, $<$, $>$, $<=$, $>=$, $==$, $!=$) имеют единичную трудоёмкость: 1;
2. операции умножения, деления и остатка от деления ($*$, $/$, $\%$, $*=$, $/=$, $\%=$) имеют трудоёмкость: 2;
3. трудоёмкость условного оператора определяется как сумма трудоёмкости вычисления условия и трудоёмкости выполняемой ветви:

$$f_{\text{if}} = f_{\text{усл}} + \begin{cases} f_{\text{ветви}}, & \text{ветвь выполняется}, \\ 0, & \text{ветвь не выполняется}; \end{cases} \tag{4.1}$$

4. трудоёмкость цикла `for` с числом итераций K вычисляется с учётом инициализации счётчика, проверки условия продолжения цикла на каждой итерации и шага инкремента:

$$f_{\text{for}} = f_{\text{иниц}} + f_{\text{усл}} + K \cdot (f_{\text{усл}} + f_{\text{инкр}} + f_{\text{тела}}). \tag{4.2}$$

Для заголовка `for (size_t i = 0; i < N; ++i)`:

$$f_{\text{for}} = 1 + 1 + K \cdot (1 + 1 + f_{\text{тела}}) = 2 + K \cdot (2 + f_{\text{тела}}). \tag{4.3}$$

Если условие цикла содержит целочисленное деление, например `k < N / 2`, то $f_{\text{усл}} = 3$. В этом случае:

$$f_{\text{for}} = 1 + 3 + K \cdot (3 + 1 + f_{\text{тела}}) = 4 + K \cdot (4 + f_{\text{тела}}); \tag{4.4}$$

5. затраты на вызовы функций, динамическое выделение памяти под матрицы и получение их размеров в оценку не включаются.

Обозначение: $H = \lfloor N/2 \rfloor$. Для квадратных матриц размера $N \times N$ при

чётных значениях N $H = N/2$, а при нечётных — $H = (N-1)/2$.

4.2 Оценка трудоёмкости алгоритмов

4.2.1 Стандартный алгоритм умножения матриц

Тело цикла по k содержит оператор:

```
C[i][j] += A[i][k] * B[k][j];
```

Трудоёмкость тела цикла по k: $f_{\text{тела_}k} = 6 + 2 + 1 = 9$.

Условие цикла `k < N` ($f_{\text{усл}} = 1$), число итераций равно N. Затраты на выполнение цикла по k:

$$f_k = 2 + N \cdot (2 + 9) = 11N + 2. \tag{4.5}$$

Цикл по j выполняется Q раз с условием ($f_{\text{усл}} = 1$):

$$f_j = 2 + Q \cdot (2 + f_k) = 2 + Q \cdot (11N + 4) = 11NQ + 4Q + 2. \tag{4.6}$$

Цикл по i выполняется M раз с условием ($f_{\text{усл}} = 1$):

$$f_{\text{станд}}(M, N, Q) = 2 + M \cdot (2 + f_j) = 11MNQ + 4MQ + 4M + 2. \tag{4.7}$$

Для квадратных матриц ($M = N = Q$) трудоёмкость стандартного алгоритма составляет:

$$f_{\text{станд}}(N) = 11N^3 + 4N^2 + 4N + 2 = O(N^3). \tag{4.8}$$

4.2.2 Базовый алгоритм Винограда

Трудоёмкость алгоритма складывается из четырёх этапов:

$$f_{\text{вин}} = f_{row} + f_{col} + f_{\text{осн}} + f_{\text{нечёт}}. \tag{4.9}$$

При вычислении вектора row тело внутреннего цикла:

```
row[i] += A[i][2 * j] * A[i][2 * j + 1];
```

Итого $f_{\text{тела_}row} = 1 + 4 + 1 + 4 + 2 + 1 = 13$. Внутренний цикл имеет условие `j < N / 2` с делением ($f_{\text{усл}} = 3$) и совершает H итераций:

$$f_{j_row} = 4 + H \cdot (4 + 13) = 17H + 4. \tag{4.10}$$

Внешний цикл по i (M итераций, условие `i < M`, $f_{\text{усл}} = 1$):

$$f_{row} = 2 + M \cdot (2 + 17H + 4) = 17MH + 6M + 2. \tag{4.11}$$

Структура вычисления вектора col аналогична вектору row: внешний цикл по i совершает Q итераций, внутренний — H итераций с условием `j < N / 2` ($f_{\text{тела_}col} = 13$, $f_{\text{усл}} = 3$):

$$f_{col} = 17QH + 6Q + 2. \tag{4.12}$$

В основной части в цикле по j выполняется инициализация:

```
C[i][j] = -row[i] - col[j];
```

Итого $f_{\text{иниц_}C} = 7$.

Тело цикла по k:

```
C[i][j] += (A[i][2 * k] + B[2 * k + 1][j]) * (A[i][2 * k + 1] +
                                              B[2 * k][j]);
```

Итого $f_{\text{тела_}k} = 2 + 8 + 2 + 8 + 2 + 2 + 1 = 25$.

Цикл по k выполняется H раз с условием `k < N / 2` ($f_{\text{усл}} = 3$):

$$f_{k_\text{осн}} = 4 + H \cdot (4 + 25) = 29H + 4. \tag{4.13}$$

Цикл по j (Q итераций, $f_{\text{усл}} = 1$):

$$f_{j_\text{осн}} = 2 + Q \cdot (2 + f_{\text{иниц_}C} + f_{k_\text{осн}}) = 2 + Q \cdot (2 + 7 + 29H + 4) = 29QH + 13Q + 2. \tag{4.14}$$

Цикл по i (M итераций, $f_{\text{усл}} = 1$):

$$f_{\text{осн}} = 2 + M \cdot (2 + 29QH + 13Q + 2) = 29MQH + 13MQ + 4M + 2. \tag{4.15}$$

Затраты на проверку условия `N % 2 == 1`: $f_{\text{усл_нечёт}} = 3$. При чётном N ветвь не выполняется: $f_{\text{нечёт}} = 3$. При нечётном N выполняется цикл с телом `C[i][j] += A[i][N - 1] * B[N - 1][j];` ($f_{\text{тела}} = 11$, $f_j = 13Q + 2$, $f_i = 13MQ + 4M + 2$):

$$f_{\text{нечёт}} = 3 + 13MQ + 4M + 2 = 13MQ + 4M + 5. \tag{4.16}$$

Трудоёмкость базового алгоритма Винограда:

$$f_{\text{вин}} = 29MQH + 17H(M + Q) + 13MQ + 10M + 6Q + 6 + f_{\text{нечёт}}. \tag{4.17}$$

Для квадратных матриц ($M = N = Q$):

— при чётном N ($H = N/2$, $f_{\text{нечёт}} = 3$):

$$\begin{aligned} f_{\text{вин_чёт}}(N) &= 29N^2 \cdot \frac{N}{2} + 17 \cdot \frac{N}{2} \cdot 2N + 13N^2 + 16N + 9 \\ &= 14{,}5N^3 + 30N^2 + 16N + 9; \end{aligned} \tag{4.18}$$

— при нечётном N ($H = (N - 1)/2$, $f_{\text{нечёт}} = 13N^2 + 4N + 5$):

$$\begin{aligned} f_{\text{вин_нечёт}}(N) &= 29N^2 \cdot \frac{N-1}{2} + 17 \cdot \frac{N-1}{2} \cdot 2N + 26N^2 + 20N + 11 \\ &= 14{,}5N^3 + 28{,}5N^2 + 3N + 11. \end{aligned} \tag{4.19}$$

4.2.3 Оптимизированный алгоритм Винограда

В оптимизированной версии применены две модификации:

1. вынесение `size_t half_n = N / 2;` перед циклами. Затраты: 3. Условия всех внутренних циклов принимают вид `k < half_n`, ($f_{\text{усл}} = 1$), что исключает операцию деления на каждой итерации;
2. вычисление вспомогательного индекса `size_t two_k = k << 1;` в начале каждой итерации внутреннего цикла (1 сдвиг + 1 присваивание = 2) и его повторное использование.

Для векторов row и col внутреннее тело цикла: вычисление `two_k` (2) и накопление `row[i] += A[i][two_k] * A[i][two_k + 1];`. Итого $f_{\text{тела}} = 2 + 9 = 11$. Цикл по k с условием ($f_{\text{усл}} = 1$): $f_k = 2 + H \cdot (2 + 11) = 13H + 2$.

Внешний цикл по строкам:

$$f_{row_\text{опт}} = 2 + M \cdot (2 + 13H + 2) = 13MH + 4M + 2. \tag{4.20}$$

Аналогично для вектора col:

$$f_{col_\text{опт}} = 13QH + 4Q + 2. \tag{4.21}$$

В основной части внутреннее тело цикла: вычисление `two_k` (2) и выражение:

```
C[i][j] += (A[i][two_k] + B[two_k + 1][j]) * (A[i][two_k + 1] +
                                              B[two_k][j]);
```

Итого $f_{\text{тела}} = 2 + (8 + 2 + 2 + 2 + 1) = 19$.

Цикл по k с условием ($f_{\text{усл}} = 1$):

$$f_{k_\text{осн_опт}} = 2 + H \cdot (2 + 19) = 21H + 2. \tag{4.22}$$

С учётом $f_{\text{иниц}_C} = 7$ цикл по j даёт:

$$f_{j_\text{осн_опт}} = 2 + Q \cdot (2 + 7 + 21H + 2) = 21QH + 11Q + 2. \tag{4.23}$$

Внешний цикл по i:

$$f_{\text{осн_опт}} = 2 + M \cdot (2 + 21QH + 11Q + 2) = 21MQH + 11MQ + 4M + 2. \tag{4.24}$$

Суммирование компонентов вместе с предварительным расчётом $f_{\text{кэш}} = 3$ и этапом нечётной коррекции $f_{\text{нечёт}}$:

$$f_{\text{опт}} = 21MQH + 13H(M + Q) + 11MQ + 8M + 4Q + 9 + f_{\text{нечёт}}. \tag{4.25}$$

Для квадратных матриц ($M = N = Q$):

— при чётном N ($H = N/2$, $f_{\text{нечёт}} = 3$):

$$f_{\text{опт_чёт}}(N) = 10{,}5N^3 + 24N^2 + 12N + 12; \tag{4.26}$$

— при нечётном N ($H = (N-1)/2$, $f_{\text{нечёт}} = 13N^2 + 4N + 5$):

$$f_{\text{опт_нечёт}}(N) = 10{,}5N^3 + 26{,}5N^2 + 3N + 14. \tag{4.27}$$

4.2.4 Сводная аналитическая оценка

В таблице 4.1 приведены полученные аналитические оценки трудоёмкости для всех трёх исследуемых реализаций на квадратных матрицах $N \times N$.

Таблица 4.1 — Теоретическая трудоёмкость алгоритмов умножения матриц

Алгоритм	**Чётный размер N**	**Нечётный размер N**	**Сложность**
Стандартный	$11N^3 + 4N^2 + 4N + 2$	$11N^3 + 4N^2 + 4N + 2$	$O(N^3)$
Винограда (базовый)	$14{,}5N^3 + 30N^2 + 16N + 9$	$14{,}5N^3 + 28{,}5N^2 + 3N + 11$	$O(N^3)$
Винограда (оптим.)	$10{,}5N^3 + 24N^2 + 12N + 12$	$10{,}5N^3 + 26{,}5N^2 + 3N + 14$	$O(N^3)$

Согласно введённой модели вычислений, базовый алгоритм Винограда оказывается более трудоёмким по сравнению со стандартным алгоритмом ($14{,}5 \cdot N^3$ против $11 \cdot N^3$ при старшем члене): двукратное сокращение количества умножений элементов матрицы нивелируется многократным повторным умножением $2 \cdot k$ при индексации и повторным делением $N/2$ в условиях циклов. Применение оптимизаций варианта (побитовый сдвиг и кэширование $N/2$) снижает коэффициент при кубическом члене с $14{,}5$ до $10{,}5$, делая оптимизированный алгоритм Винограда наименее трудоёмким из всех трёх версий.

4.3 Характеристики устройства

Экспериментальные замеры процессорного времени выполнялись на вычислительном комплексе со следующими характеристиками:

— процессор: Intel Core i5-1335U, 10 ядер, 12 потоков, тактовая частота до 4,60 ГГц;

— оперативная память: 16 ГБ DDR4, частота 3200 МГц;

— операционная система: Linux Ubuntu 24.04.4 LTS;

— компилятор: GCC 13.3.0, флаги сборки: `-std=c++20 -Wall -Wextra -Wpedantic -O0`.

4.4 Инструменты замера процессорного времени

Для измерения процессорного времени работы алгоритмов использовалась функция `clock()` из заголовочного файла `<ctime>` [4].

Перевод полученного значения в секунды осуществляется делением на константу `CLOCKS_PER_SEC`:

$$t = \frac{\text{clock}_{\text{end}} - \text{clock}_{\text{start}}}{\text{CLOCKS_PER_SEC} \cdot K}, \tag{4.28}$$

где K — количество повторов умножения одной и той же пары матриц на текущем размере.

4.5 Результаты массированных замеров

Массированные замеры проводились для квадратных матриц в диапазоне размеров $N \in [10, 500]$ раздельно для чётных и нечётных размерностей. Результаты замеров сведены в таблицы 4.2 и 4.3.

Таблица 4.2 — Процессорное время умножения матриц чётных размеров, с

Размер N	**Стандартный**	**Виноград (базовый)**	**Виноград (оптим.)**
10	0,000029	0,000028	0,000028
20	0,000195	0,000119	0,000136
40	0,000501	0,000430	0,000427
60	0,001311	0,001398	0,001395
80	0,004431	0,003607	0,003615
100	0,005944	0,006139	0,006177
150	0,019987	0,020349	0,020742
200	0,047282	0,047742	0,048545
250	0,092496	0,093359	0,094982
300	0,161527	0,159662	0,163999
400	0,377031	0,374348	0,394725
500	0,734502	0,756302	0,766590

Таблица 4.3 — Процессорное время умножения матриц нечётных размеров, с

Размер N	**Стандартный**	**Виноград (базовый)**	**Виноград (оптим.)**
11	0,000010	0,000011	0,000012
21	0,000061	0,000083	0,000074
41	0,000423	0,000462	0,000472
61	0,001371	0,001473	0,001461
81	0,003182	0,003370	0,003348
101	0,006288	0,006358	0,006437
151	0,020745	0,020833	0,021056
201	0,047889	0,048252	0,050058
251	0,093132	0,093312	0,101094
301	0,164249	0,174337	0,168708
401	0,381010	0,395288	0,393645
501	0,741757	0,764204	0,772992

Графики зависимости процессорного времени от линейного размера квадратных матриц для чётных и нечётных размеров приведены на рисунках 4.1 и 4.2.

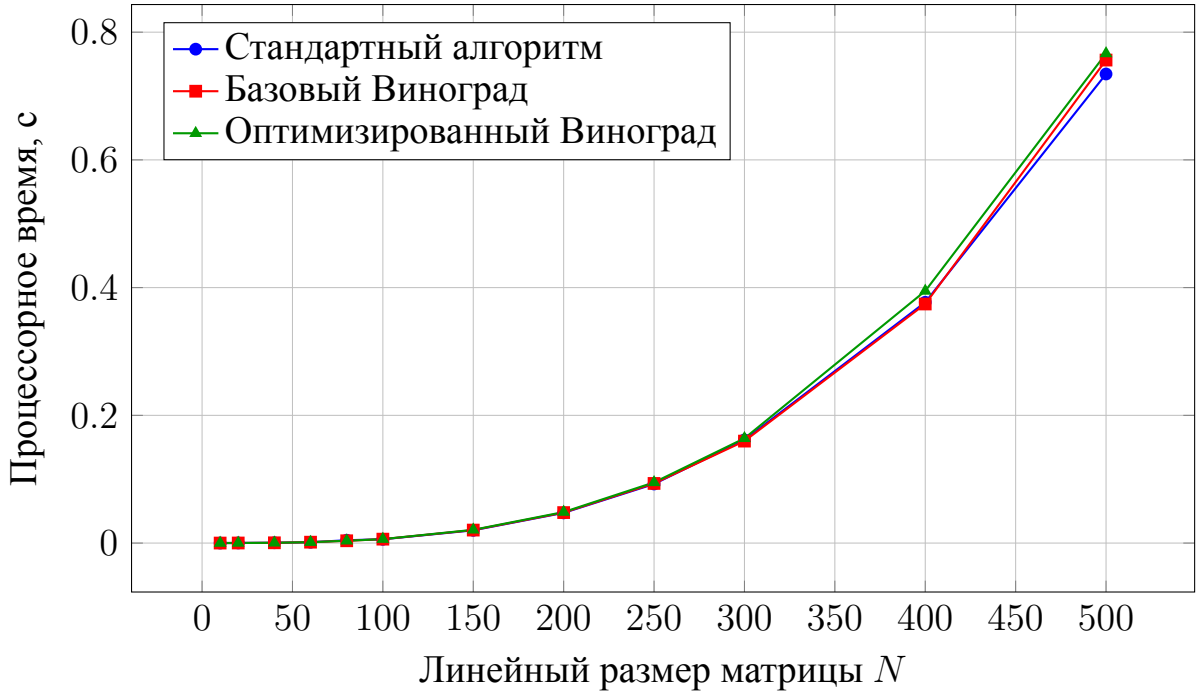


Рисунок 4.1 — Зависимость процессорного времени от размера матрицы N (чётные размеры)

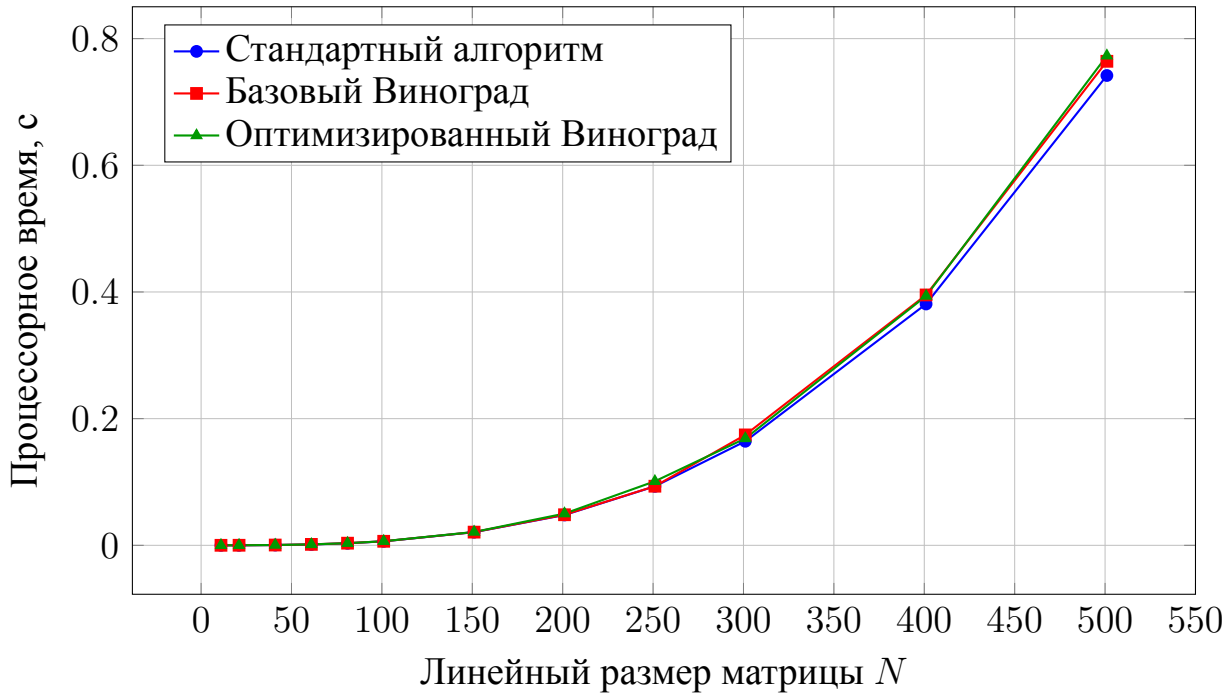


Рисунок 4.2 — Зависимость процессорного времени от размера матрицы N (нечётные размеры)

4.6 Сравнительный анализ

Для сопоставления результатов было проведено сравнение относительной трудоёмкости по теоретической модели $f_{\text{отн}}$ и замеренному процессорному времени $t_{\text{отн}}$ относительно стандартного алгоритма:

$$f_{\text{отн}} = \frac{f_X(N)}{f_{\text{станд}}(N)}, \quad t_{\text{отн}} = \frac{t_X(N)}{t_{\text{станд}}(N)}. \tag{4.29}$$

В таблице 4.4 сведены значения для чётного размера $N = 500$.

Таблица 4.4 — Сравнение алгоритмов со стандартным при $N = 500$

Алгоритм	**Трудоёмкость**	$f_{\text{отн}}$	**Время, с**	$t_{\text{отн}}$
Стандартный	1 376 002 002	1,000	0,7345	1,000
Виноград (базовый)	1 820 008 009	1,323	0,7563	1,030
Виноград (оптим.)	1 318 506 012	0,958	0,7666	1,044

Анализ теоретических и экспериментальных данных позволяет сделать следующие выводы:

— При увеличении размера матриц в 2 раза (например, со 100 до 200 и с 250 до 500) время работы всех реализаций возрастает примерно в 8 раз ($0{,}0925$ с $\rightarrow$ $0{,}7345$ с, то есть в $7{,}94$ раза), что подтверждает кубический порядок роста $O(N^3)$ ($2^3 = 8$).

— Согласно теоретической модели вычислений, базовый алгоритм Винограда более трудоёмок, чем стандартный ($f_{\text{отн}} \approx 1{,}32$), что подтверждается экспериментально: стандартный алгоритм выполняется быстрее базовой версии Винограда ($0{,}7345$ с против $0{,}7563$ с).

— Разница по времени между всеми тремя реализациями при $N = 500$ составляет не более $4{,}4\%$ ($0{,}7345$ с, $0{,}7563$ с и $0{,}7666$ с), а оптимизированная версия показывает незначительное отставание от базовой.

— Качественных различий между чётными и нечётными размерами матриц не обнаружено. Поправка на нечётность N имеет порядок $O(N^2)$ и при больших размерах ($N \geq 100$) вносит пренебрежимо малый вклад на фоне кубической составляющей $O(N^3)$.

4.7 Вывод

В исследовательской части была разработана модель вычислений, выполнен теоретический расчёт трудоёмкости всех исследуемых алгоритмов, проведены экспериментальные замеры процессорного времени и выполнен сравнительный анализ алгоритмов умножения матриц.

ЗАКЛЮЧЕНИЕ

В процессе выполнения лабораторной работы поставленная цель была достигнута: проведён сравнительный анализ стандартного алгоритма умножения матриц, базового алгоритма Винограда и его оптимизированной модификации по теоретической трудоёмкости и процессорному времени выполнения.

Все задачи работы решены:

— описана решаемая задача умножения матриц согласно варианту;

— разработаны стандартный алгоритм умножения матриц, алгоритм Винограда и оптимизированный по варианту алгоритм Винограда;

— описаны разработанные алгоритмы схемами алгоритмов;

— сформулированы требования к разрабатываемому программному обеспечению;

— описаны средства реализации алгоритмов;

— программно реализованы все разработанные алгоритмы умножения матриц;

— проведено функциональное тестирование реализаций;

— сформирована модель вычислений и выполнен аналитический расчёт трудоёмкости всех исследуемых алгоритмов;

— описаны инструменты замера процессорного времени;

— выполнены серии массированных замеров процессорного времени выполнения реализаций для квадратных матриц чётных и нечётных порядков;

— проведён сравнительный анализ полученных теоретических оценок трудоёмкости и экспериментальных временных характеристик.

В результате исследования установлено, что все рассмотренные алгоритмы обладают кубической асимптотической сложностью $O(N^3)$. В соответствии с теоретической моделью вычислений базовый алгоритм Винограда характеризуется большей трудоёмкостью ($14{,}5N^3$) по сравнению со стандартным ($11N^3$) по старшему члену, однако применение оптимизаций снижает её до $10{,}5N^3$. Экспериментальные замеры подтвердили теоретический порядок роста $O(N^3)$ и отсутствие качественных различий между чётными и нечётными порядками матриц.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Кормен, Т. Алгоритмы: построение и анализ : пер. с англ. / Т. Кормен, Ч. Лейзерсон, Р. Ривест, К. Штайн. — 3-е изд. — М. : ООО «И.Д. Вильямс», 2013. — 1328 с.

2. Winograd, S. A New Algorithm for Inner Product / S. Winograd // IEEE Transactions on Computers. — 1968. — Vol. C-17, no. 7. — P. 693–694.

3. ISO/IEC 14882:2020. Programming languages — C++. — International Organization for Standardization, 2020.

4. std::clock — cppreference.com [Электронный ресурс]. — Режим доступа: `https://en.cppreference.com/w/cpp/chrono/c/clock` (дата обращения: 20.10.2026).